Занятие 8: Работа с базой данных

Подключение, драйверы, SQL, ORM

Слайд 1: Путь данных от HTTP до БД

Полный цикл запроса:

```
1. HTTP Request
   ↓
2. Web Server (Nginx)
   ↓
3. Application (Python/Flask)
   ↓
4. Database Driver (psycopg2/mysql-connector)
   ↓
5. Database Server (PostgreSQL/MySQL)
   ↓
6. Response back through the chain
```

Ключевые компоненты:

- **Application** - бизнес-логика
- **Driver** - библиотека для связи с БД
- **Database Server** - хранение данных

Слайд 2: Подключение к БД

Параметры подключения:

- **Host** - адрес сервера БД
- **Port** - порт (PostgreSQL: 5432, MySQL: 3306)
- **Database** - имя базы данных
- **User** - имя пользователя
- **Password** - пароль

Пример строки подключения:

```
postgresql://user:password@localhost:5432/mydb
mysql://user:password@localhost:3306/mydb
```

Пример на Python (psycopg2):

```python
import psycopg2

conn = psycopg2.connect(
    host="localhost",
    port=5432,
    database="mydb",
    user="postgres",
    password="password"
)
```

Слайд 3: Драйверы БД

Что такое драйвер?

Драйвер - библиотека, которая позволяет приложению общаться с БД.

Популярные драйверы:

БД	Python драйвер	Node.js драйвер
PostgreSQL	psycopg2, asyncpg	pg
MySQL	mysql-connector-python, PyMySQL	mysql2
SQLite	sqlite3 (встроенный)	better-sqlite3

Пример установки:

```
pip install psycopg2-binary
pip install mysql-connector-python
```

Слайд 4: Выполнение SQL запросов

Шаги выполнения запроса:

1. Создать подключение (Connection)
2. Создать курсор (Cursor)
3. Выполнить запрос (execute)
4. Получить результат (fetch)
5. Закрыть курсор и подключение

Пример:

```
import psycopg2

conn = psycopg2.connect("...")
cursor = conn.cursor()

cursor.execute("SELECT * FROM users")
results = cursor.fetchall()

cursor.close()
conn.close()
```

Слайд 5: INSERT - вставка данных

Пример вставки пользователя:

```
cursor.execute(
    "INSERT INTO users (name, email, age) VALUES (%s, %s, %s)",
    ("John Doe", "john@example.com", 30)
)
conn.commit()
```

Важно:

- Используйте параметризованные запросы (%s или ?)
- Не подставляйте значения напрямую (SQL injection!)
- Вызовите commit() для сохранения изменений

Плохо (SQL injection):

```
name = input("Name: ")
cursor.execute(f"INSERT INTO users (name) VALUES ('{name}')")  # НЕ ДЕЛАЙТЕ ТАК!
```

Слайд 6: SELECT - чтение данных

Примеры запросов:

```
# Все записи
cursor.execute("SELECT * FROM users")
users = cursor.fetchall()
# [(1, 'John', 'john@example.com', 30), ...]

# Одна запись
cursor.execute("SELECT * FROM users WHERE id = %s", (1,))
user = cursor.fetchone()
# (1, 'John', 'john@example.com', 30)

# Определённые колонки
cursor.execute("SELECT name, email FROM users")
names = cursor.fetchall()
```

fetchone() vs fetchall():

- `fetchone()` - одна запись (или None)
- `fetchall()` - все записи (список)
- `fetchmany(n)` - n записей

Слайд 7: UPDATE и DELETE

UPDATE - обновление:

```
cursor.execute(
    "UPDATE users SET age = %s WHERE id = %s",
    (31, 1)
)
conn.commit()
```

DELETE - удаление:

```
cursor.execute("DELETE FROM users WHERE id = %s", (1,))
conn.commit()
```

Важно:

- Всегда используйте WHERE
- Проверяйте количество затронутых строк
- Используйте транзакции для связанных изменений

Проверка затронутых строк:

```
cursor.execute("UPDATE users SET age = 30 WHERE id = %s", (1,))
print(f"Updated {cursor.rowcount} rows")
```

Слайд 8: Обработка ошибок

Типичные ошибки:

```
try:
    conn = psycopg2.connect("...")
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM users")
    results = cursor.fetchall()

except psycopg2.OperationalError as e:
    print(f"Connection error: {e}")

except psycopg2.ProgrammingError as e:
    print(f"SQL syntax error: {e}")

except Exception as e:
    print(f"Unexpected error: {e}")

finally:
    if 'cursor' in locals():
        cursor.close()
    if 'conn' in locals():
        conn.close()
```

Используйте context manager:

```
with psycopg2.connect("...") as conn:
    with conn.cursor() as cursor:
        cursor.execute("SELECT * FROM users")
        results = cursor.fetchall()
# Автоматически закроет cursor и conn
```

Слайд 9: SQL Injection

Что такое SQL Injection?

Атака, при которой злоумышленник внедряет SQL код в запрос.

Пример уязвимости:

```
email = input("Email: ")
query = f"SELECT * FROM users WHERE email = '{email}'"
cursor.execute(query)
```

Атака:

```
Email: ' OR '1'='1
Query: SELECT * FROM users WHERE email = '' OR '1'='1'
Результат: Все пользователи!
```

Защита - параметризованные запросы:

```
email = input("Email: ")
cursor.execute("SELECT * FROM users WHERE email = %s", (email,))
```

Слайд 10: Параметризованные запросы

Почему это безопасно?

Драйвер экранирует параметры отдельно от SQL:

```
# Безопасно
cursor.execute(
    "SELECT * FROM users WHERE email = %s",
    ("' OR '1'='1",)
)
# Будет искать пользователя с email "' OR '1'='1"
```

Разные синтаксисы:

Драйвер	Синтаксис
psycopg2	%s
mysql-connector	%s
sqlite3	?
asyncpg	$1, $2

Пример с несколькими параметрами:

```
cursor.execute(
    "INSERT INTO users (name, email, age) VALUES (%s, %s, %s)",
    ("John", "john@example.com", 30)
)
```

Слайд 11: JOIN - объединение таблиц

Пример схемы:

```
users:      orders:
id          id
name        user_id
email       product
            total
```

INNER JOIN:

```
cursor.execute("""
    SELECT users.name, orders.product, orders.total
    FROM users
    INNER JOIN orders ON users.id = orders.user_id
""")
```

LEFT JOIN:

```
cursor.execute("""
    SELECT users.name, orders.product
    FROM users
    LEFT JOIN orders ON users.id = orders.user_id
""")
# Вернёт всех пользователей, даже без заказов
```

Слайд 12: Миграции

Что такое миграции?

Миграции - управляемые изменения схемы БД.

Зачем нужны:

- Версионирование схемы БД
- Применение изменений в команде
- Откат изменений

Пример миграции:

```sql
-- migration_001_create_users.sql
CREATE TABLE users (
    id SERIAL PRIMARY KEY,
    name VARCHAR(50) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL,
    age INTEGER CHECK (age >= 18),
    created_at TIMESTAMP DEFAULT NOW()
);
```

```sql
-- migration_002_add_phone.sql
ALTER TABLE users ADD COLUMN phone VARCHAR(20);
```

Слайд 13: Инструменты миграций

Популярные инструменты:

Alembic (Python):

```
pip install alembic
alembic init alembic
alembic revision --autogenerate -m "Add users table"
alembic upgrade head
alembic downgrade -1
```

Flyway (Java/Go/Python):

```
-- V1__Create_users_table.sql
CREATE TABLE users (...);

-- V2__Add_phone_column.sql
ALTER TABLE users ADD COLUMN phone VARCHAR(20);
```

Пример Alembic:

```
def upgrade():
    op.create_table('users',
        sa.Column('id', sa.Integer(), primary_key=True),
        sa.Column('name', sa.String(50)),
        sa.Column('email', sa.String(100))
    )

def downgrade():
    op.drop_table('users')
```

Слайд 14: ORM - что это?

ORM (Object-Relational Mapping)

Преобразование объектов в записи БД и наоборот.

Пример без ORM:

```
cursor.execute("SELECT * FROM users WHERE id = %s", (1,))
row = cursor.fetchone()
user = {'id': row[0], 'name': row[1], 'email': row[2]}
```

Пример с ORM (SQLAlchemy):

```
user = session.query(User).filter_by(id=1).first()
print(user.name)
```

Преимущества ORM:

- Работаем с объектами, а не SQL
- Автоматическая генерация SQL
- Защита от SQL injection
- Миграции через модели

Слайд 15: SQLAlchemy пример

Определение модели:

```
from sqlalchemy import create_engine, Column, Integer, String
from sqlalchemy.orm import declarative_base, sessionmaker

Base = declarative_base()

class User(Base):
    __tablename__ = 'users'

    id = Column(Integer, primary_key=True)
    name = Column(String(50))
    email = Column(String(100), unique=True)
    age = Column(Integer)

# Создание таблицы
engine = create_engine('postgresql://...')
Base.metadata.create_all(engine)

Session = sessionmaker(bind=engine)
session = Session()
```

CRUD операции:

```
# Create
user = User(name="John", email="john@example.com", age=30)
session.add(user)
session.commit()

# Read
user = session.query(User).filter_by(id=1).first()
```

Слайд 16: Разбор примера db_operations.py

Структура примера:

```
class Database:
    def __init__(self):
        # Connection Pool - кэш подключений
        self.connection_pool = psycopg2.pool.SimpleConnectionPool(
            minconn=1, maxconn=10, ...
        )

    @contextmanager
    def get_connection(self):
        # Автоматическое возврат подключения в пул
        conn = self.connection_pool.getconn()
        try:
            yield conn
        finally:
            self.connection_pool.putconn(conn)
```

Поток выполнения:

1. Приложение запрашивает подключение из пула
2. Если есть свободное - выдаётся, если нет - создаётся новое
3. После использования - возвращается в пул
4. Максимальное подключений - 10 (maxconn)

Преимущества:

- Нет задержки на создание подключения
- Контроль количества подключений к БД

Слайд 17: Разбор примера concurrency_consistency.py

Race Condition - что происходит:

```
# Поток 1: читает balance = 1000
# Поток 2: читает balance = 1000 (одновременно)
# Поток 1: balance = 1000 - 100 = 900
# Поток 2: balance = 1000 - 100 = 900
# Результат: баланс = 900 (должен быть 800!)
```

Pessimistic Locking - решение:

```
SELECT balance FROM accounts WHERE id = 1 FOR UPDATE
# Строка блокируется до конца транзакции
# Поток 2 ждёт, пока поток 1 не завершит
```

Оптимистическая блокировка - решение:

```
UPDATE accounts SET balance = 900, version = 2
WHERE id = 1 AND version = 1
# Если rowcount = 0 - кто-то изменил версию
# Повторить попытку с новой версией
```

Запуск в Docker:

```
docker-compose up -d
# Автоматически запускает все примеры конкурентности
# Сравнивает результаты разных подходов
```

Слайд 18: Асинхронный доступ к БД с Docker

Синхронный vs Асинхронный:

```
# Синхронный (psycopg2)
def get_user(user_id):
    conn = psycopg2.connect("...")
    cursor = conn.cursor()
    cursor.execute("SELECT * FROM users WHERE id = %s", (user_id,))
    return cursor.fetchone()
# Блокирует поток

# Асинхронный (asyncpg)
async def get_user(user_id):
    conn = await asyncpg.connect("...")
    row = await conn.fetchrow("SELECT * FROM users WHERE id = $1", user_id)
    await conn.close()
    return row
# Не блокирует
```

docker-compose.yml для asyncpg:

```
version: '3.8'

services:
  postgres:
    image: postgres:15-alpine
    environment:
      POSTGRES_USER: webprog
      POSTGRES_PASSWORD: webprog123
      POSTGRES_DB: webprog
    ports:
      - "5432:5432"
    volumes:
```

Слайд 19: Конкурентность и консистентность

Проблемы конкурентного доступа:

Race Condition (Гонка данных):

```
# Два потока читают баланс одновременно
balance = get_balance(account_id)
# Симуляция задержки
time.sleep(0.1)
# Обновляют одно и то же значение
update_balance(account_id, balance - amount)
# Результат: потеря обновлений
```

Pessimistic Locking (Пессимистическая блокировка):

```
# SELECT FOR UPDATE блокирует строку
cursor.execute(
    "SELECT balance FROM accounts WHERE id = %s FOR UPDATE",
    (account_id,)
)
balance = cursor.fetchone()[0]
# Другие транзакции ждут освобождения блокировки
```

Optimistic Locking (Оптимистическая блокировка):

```
# Использует version column
cursor.execute("""
    UPDATE accounts
    SET balance = %s, version = version + 1
    WHERE id = %s AND version = %s
""", (new_balance, account_id, current_version))
```

Слайд 19: Upsert и Savepoints

Upsert (INSERT ... ON CONFLICT):

```
cursor.execute("""
    INSERT INTO products (sku, name, quantity)
    VALUES (%s, %s, %s)
    ON CONFLICT (sku) DO UPDATE SET
        name = EXCLUDED.name,
        quantity = EXCLUDED.quantity
""", (sku, name, quantity))
```

Savepoints (частичный откат):

```
cursor.execute("INSERT INTO transactions ...")
cursor.execute("SAVEPOINT my_savepoint")

cursor.execute("INSERT INTO transactions ...")

# Откат к savepoint, но не всей транзакции
cursor.execute("ROLLBACK TO SAVEPOINT my_savepoint")

cursor.execute("INSERT INTO transactions ...")
cursor.connection.commit()
```

Слайд 20: Уровни изоляции транзакций

Уровни изоляции в PostgreSQL:

Уровень	Описание	Грязное чтение	Неповторяемое чтение	Фантомное чтение
READ COMMITTED	По умолчанию	Нет	Да	Да
REPEATABLE READ	Снимок данных	Нет	Нет	Да
SERIALIZABLE	Полная изоляция	Нет	Нет	Нет

Установка уровня:

```
cursor.execute("SET TRANSACTION ISOLATION LEVEL REPEATABLE READ")
```

Когда использовать:

- READ COMMITTED - большинство случаев
- REPEATABLE READ - отчёты, аналитика
- SERIALIZABLE - критические финансовые операции

Слайд 22: Batch Insert

Одиночная вставка (медленно):

```
for item in items:
    cursor.execute("INSERT INTO table VALUES (%s)", (item,))
# 100 запросов к БД
```

Batch insert (быстро):

```
data = [(item1,), (item2,), ...]
cursor.executemany("INSERT INTO table VALUES (%s)", data)
# 1 запрос к БД
```

Производительность:

- Batch insert обычно в 10-100x быстрее
- Меньше сетевых задержек
- Меньше overhead на транзакции

Проверка через Docker:

```
# Сравнение времени выполнения
time python code/concurrency_consistency.py
```

Слайд 23: Практическое задание

Задание 1: Подключение к БД

- Создать подключение к PostgreSQL/MySQL
- Выполнить простой SELECT запрос
- Обработать ошибки подключения

Задание 2: CRUD операции

- Реализовать INSERT, SELECT, UPDATE, DELETE
- Использовать параметризованные запросы
- Проверить количество затронутых строк

Задание 3: JOIN

- Создать две связанные таблицы
- Выполнить INNER JOIN и LEFT JOIN
- Объяснить разницу

Задание 4: Миграции

- Создать миграцию для таблицы
- Применить миграцию
- Создать миграцию для изменения
- Откатить миграцию

Слайд 24: Вопрос для собеседования

Вопрос: Объясните безопасную запись и чтение данных из БД.

Ожидаемый ответ:

1. Использовать параметризованные запросы для защиты от SQL injection
2. Использовать транзакции для атомарности операций
3. Обрабатывать ошибки (connection error, syntax error)
4. Использовать connection pooling для производительности
5. Закрывать подключения (или использовать context manager)
6. Валидировать данные перед записью
7. Использовать миграции для управления схемой